Ministry of Education of Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

Laboratory work 5: Empirical Analysis of Prim and Kruskal Algorithms

Author:
st. gr. FAF-241 Moisei Daniel
Verified:
asist. univ. Fiștic Cristofor

Chișinău - 2026

Contents

ALGORITHM ANALYSIS **2**
Objective 2
Tasks 2
Introduction 2

ALGORITHM IMPLEMENTATIONS **3**
Prim (Classic) 3
Prim (Optimized) 5
Kruskal (Classic) 7
Kruskal (Optimized) 9

EMPIRICAL ANALYSIS BY GRAPH DENSITY **12**
Very Sparse Graph 12
Sparse Graph 15
Dense Graph 17
Very Dense Graph 19

OVERALL RESULTS **21**

CONCLUSION **22**

REFERENCES **23**

ALGORITHM ANALYSIS

Objective

Study, implement, and empirically analyze the performance of four algorithm variants: classic and optimized versions of Prim's and Kruskal's Minimum Spanning Tree algorithms, evaluated across four graph density levels and varying input sizes.

Tasks:

1. Study the greedy algorithm design technique;
2. Implement classic and optimized versions of Prim and Kruskal using the greedy approach;
3. Perform empirical analysis of these algorithms for sparse and dense graphs;
4. Analyze how increasing the number of nodes influences the algorithms;
5. Present a graphical visualization of the collected data;
6. Compile a report summarizing the findings.

Introduction:

Minimum Spanning Tree (MST) computation is a fundamental problem in graph theory and combinatorial optimization, with applications spanning network design, cluster analysis, image segmentation, and circuit layout. In this laboratory, we implement and benchmark four algorithm variants — Prim Classic, Prim Optimized, Kruskal Classic, and Kruskal Optimized — across four structurally distinct graph densities to observe how implementation choices and graph structure jointly influence real-world performance.

Both algorithms follow the greedy paradigm: at each step they make the locally optimal choice — the cheapest edge that does not violate the MST invariant — and never revise that decision. Despite sharing this principle, they differ fundamentally in their approach. Prim grows a single connected tree by repeatedly adding the cheapest edge crossing the current cut, while Kruskal processes all edges globally in sorted order and merges disjoint components using a Union-Find data structure. While their asymptotic complexities differ depending on implementation — $O(V^2)$ for classic Prim versus $O(E \log E)$ for Kruskal — practical execution time is shaped by data structure overhead, memory access patterns, edge density, and the cost of sorting. By testing across very sparse, sparse, dense, and very dense graph forms, this study provides a comprehensive empirical picture of MST algorithm behavior beyond what asymptotic notation alone can reveal.

ALGORITHM IMPLEMENTATIONS

Prim (Classic)

The classic Prim implementation uses a linear scan over all non-MST vertices to find the minimum-weight edge crossing the cut at each step. A `bool[]` array tracks which vertices have been added to the MST, a `double[]` array stores the cheapest known edge weight connecting each vertex to the growing tree, and an `int[]` array records the MST parent of each vertex. This approach mirrors the original formulation of the algorithm and prioritizes simplicity over raw performance.

Pseudocode:

Listing 1: Prim Classic

```
Prim(graph, source):
    minEdge = double[graph.VertexCount]  // fill with infinity
    parent  = int[graph.VertexCount]     // fill with -1
    inMST   = bool[graph.VertexCount]
    minEdge[source] = 0.0
    totalWeight = 0

    for iter = 0 to VertexCount - 1:
        u = vertex with minimum minEdge[u] where not inMST[u]  // O(V) scan
        if u == -1 or minEdge[u] == infinity: break
        inMST[u] = true
        totalWeight += minEdge[u]

        for each (neighbor v, weight w) in graph.GetNeighbors(u):
            if not inMST[v] and w < minEdge[v]:
                minEdge[v] = w
                parent[v]  = u

    return totalWeight, mstEdges
```

Implementation:

```
public static class PrimClassic
{
    1 reference
    public static (double totalWeight, List<(int u, int v, double w)> edges) Run(IGraph graph, int source = 0)
    {
        int n = graph.VertexCount;
        double[] minEdge = new double[n];
        int[] parent = new int[n];
        bool[] inMST = new bool[n];

        Array.Fill(minEdge, double.PositiveInfinity);
        Array.Fill(parent, -1);
        minEdge[source] = 0.0;

        double totalWeight = 0;
        var mstEdges = new List<(int, int, double)>();

        for (int iter = 0; iter < n; iter++)
        {
            // O(V) linear scan: find non-MST vertex with minimum edge weight
            int u = -1;
            for (int i = 0; i < n; i++)
            {
                if (!inMST[i] && (u == -1 || minEdge[i] < minEdge[u]))
                    u = i;
            }
            if (u == -1 || double.IsPositiveInfinity(minEdge[u])) break;

            inMST[u] = true;
            totalWeight += minEdge[u];
            if (parent[u] != -1)
                mstEdges.Add((parent[u], u, minEdge[u]));

            // Relax edges from u
            foreach (var (v, w) in graph.GetNeighbors(u))
            {
                if (!inMST[v] && w < minEdge[v])
                {
                    minEdge[v] = w;
                    parent[v] = u;
                }
            }
        }
        return (totalWeight, mstEdges);
    }
```

Figure 1: Prim Classic implementation in C#

Complexity Facts & Performance Characteristics:

- **Complexity:** $O(V^2)$ time, $O(V)$ space.
- **Linear scan overhead:** Finding the minimum-weight non-MST vertex requires iterating over all V vertices at each of the V iterations, producing a V^2 inner loop that dominates on large graphs regardless of how many edges exist.
- **Dense graph advantage:** On very dense graphs where $E \approx V^2$, the $O(V^2)$ classic version becomes competitive with the heap-based optimized version, since the heap accumulates a large number of entries and the $O(\log V)$ extraction cost accumulates to approach the flat linear scan.
- **No GC pressure:** The flat array structure avoids heap allocations beyond the initial setup, keeping garbage collection interference minimal throughout execution.

Prim (Optimized)

The optimized Prim replaces the $O(V)$ linear scan with a binary min-heap implemented via .NET's built-in `PriorityQueue<TElement, TPriority>`. Rather than scanning all vertices, the heap always provides the minimum-weight candidate edge in $O(\log V)$ time. Stale heap entries — vertices re-inserted after a cheaper edge was found — are discarded lazily by checking the `inMST` flag on extraction.

Pseudocode:

Listing 2: Prim Optimized (Binary Min-Heap)

```
Prim(graph, source):
    minEdge = double[graph.VertexCount]  // fill with infinity
    parent  = int[graph.VertexCount]     // fill with -1
    inMST   = bool[graph.VertexCount]
    minEdge[source] = 0.0
    pq = PriorityQueue<vertex, weight>
    pq.Enqueue(source, 0.0)
    totalWeight = 0

    while pq not empty:
        (u, w) = pq.Dequeue()
        if inMST[u]: continue   // stale entry, skip
        inMST[u] = true
        totalWeight += w

        for each (neighbor v, edgeW) in graph.GetNeighbors(u):
            if not inMST[v] and edgeW < minEdge[v]:
                minEdge[v] = edgeW
                parent[v]  = u
                pq.Enqueue(v, edgeW)

    return totalWeight, mstEdges
```

Implementation:

```csharp
public static class PrimOptimized
{
    1 reference
    public static (double totalWeight, List<(int u, int v, double w)> edges) Run(IGraph graph, int source = 0)
    {
        int n = graph.VertexCount;
        double[] minEdge = new double[n];
        int[] parent = new int[n];
        bool[] inMST = new bool[n];

        Array.Fill(minEdge, double.PositiveInfinity);
        Array.Fill(parent, -1);
        minEdge[source] = 0.0;

        // PriorityQueue<vertex, weight>
        var pq = new PriorityQueue<int, double>(n);
        pq.Enqueue(source, 0.0);

        double totalWeight = 0;
        var mstEdges = new List<(int, int, double)>();

        while (pq.Count > 0)
        {
            pq.TryDequeue(out int u, out double w);

            // Stale entry: already added to MST with a cheaper edge
            if (inMST[u]) continue;

            inMST[u] = true;
            totalWeight += w;
            if (parent[u] != -1)
                mstEdges.Add((parent[u], u, w));

            foreach (var (v, edgeW) in graph.GetNeighbors(u))
            {
                if (!inMST[v] && edgeW < minEdge[v])
                {
                    minEdge[v] = edgeW;
                    parent[v] = u;
                    pq.Enqueue(v, edgeW);
                }
            }
        }

        return (totalWeight, mstEdges);
    }
}
```

Figure 2: Prim Optimized implementation in C#

Why the Optimized Version is Faster:

- **Binary heap extraction:** Replacing the $O(V)$ linear scan with an $O(\log V)$ heap extraction reduces the overall complexity from $O(V^2)$ to $O((V+E)\log V)$. This is especially significant on sparse graphs where $E \ll V^2$, as the heap stays small and each extraction is cheap.

- **Lazy deletion of stale entries:** When a cheaper edge to a vertex is found, a new entry is pushed to the heap rather than updating the existing one. Stale entries are discarded on extraction by checking the `inMST` flag. This avoids the complexity of a decrease-key operation while preserving correctness.

- **Classic vs Optimized:** On sparse and medium-density graphs, the optimized version delivers substantial speedups. On very dense graphs, the heap accumulates many stale entries and the logarithmic overhead per operation can erode the advantage, narrowing the gap with the classic linear scan.

Kruskal (Classic)

Kruskal's algorithm builds the MST by processing edges globally in non-decreasing order of weight and greedily adding each edge that connects two previously disconnected components. Cycle detection is handled by a Union-Find (Disjoint Set Union) data structure. The classic implementation uses recursive path compression without union by rank, and sorts the edge list using the standard `List.Sort()` comparison.

Pseudocode:

Listing 3: Kruskal Classic

```
Kruskal(graph):
    edges = graph.GetAllEdges()
    edges.Sort() by weight ascending          // O(E log E)

    parent = [0, 1, 2, ..., V-1]             // Union-Find with path
        compression only
    totalWeight = 0

    for each (u, v, w) in sorted edges:
        pu = Find(parent, u)
        pv = Find(parent, v)
        if pu == pv: continue                 // same component, skip

        parent[pu] = pv                       // Union (no rank)
        totalWeight += w
        add (u, v, w) to mstEdges
        if |mstEdges| == V - 1: break

    return totalWeight, mstEdges

Find(parent, x):                              // recursive path
    compression
    if parent[x] != x:
        parent[x] = Find(parent, parent[x])
    return parent[x]
```

Implementation:

```
public static class KruskalClassic
{
    1 reference
    public static (double totalWeight, List<(int u, int v, double w)> edges) Run(WeightedGraph graph)
    {
        var edges = graph.GetAllEdges();

        // Sort edges by weight - classic: simple comparison sort via LINQ
        edges.Sort((a, b) => a.w.CompareTo(b.w));

        int n = graph.VertexCount;
        int[] parent = Enumerable.Range(0, n).ToArray();

        double totalWeight = 0;
        var mstEdges = new List<(int, int, double)>();

        foreach (var (u, v, w) in edges)
        {
            int pu = Find(parent, u);
            int pv = Find(parent, v);
            if (pu == pv) continue; // same component - would form a cycle

            // Union: attach one root to the other (no rank)
            parent[pu] = pv;
            totalWeight += w;
            mstEdges.Add((u, v, w));

            if (mstEdges.Count == n - 1) break; // MST complete
        }

        return (totalWeight, mstEdges);
    }

    /// <summary>Path-compression only (no union by rank).</summary>
    3 references
    private static int Find(int[] parent, int x)
    {
        if (parent[x] != x)
            parent[x] = Find(parent, parent[x]); // recursive path compression
        return parent[x];
    }
}
```

Figure 3: Kruskal Classic implementation in C#

Complexity Facts & Performance Characteristics:

- **Complexity:** $O(E \log E)$ time dominated by sorting, $O(V + E)$ space.
- **Edge-driven runtime:** Unlike Prim, Kruskal's performance depends directly on E, not V alone. On dense graphs where $E \approx V^2$, the sort alone requires $O(V^2 \log V)$ operations, making Kruskal significantly slower than Prim on high-density inputs.
- **Union-Find without rank:** Using only path compression without union by rank means that tree heights are not bounded. While path compression alone still provides amortized near-linear performance in practice, the lack of rank control can cause taller Union-Find trees and slightly more work per `Find` call compared to the combined strategy.
- **Memory allocation for edges:** Kruskal must materialize all edges into a sortable list before processing. On dense graphs this list can contain $O(V^2)$ entries, incurring substantial memory allocation overhead that Prim, which traverses adjacency lists on the fly, entirely avoids.

Kruskal (Optimized)

The optimized Kruskal introduces two improvements over the classic version: the Union-Find structure is enhanced with both path compression and union by rank, reducing the amortized cost of `Find` to near-constant (inverse Ackermann function), and path compression is made iterative rather than recursive to eliminate call-stack overhead on long chains. The edge sort remains $O(E \log E)$ and represents the dominant cost at all densities.

Pseudocode:

Listing 4: Kruskal Optimized (Union by Rank + Iterative Path Compression)

```
Kruskal(graph):
    edges = graph.GetAllEdges()
    edges.Sort() by weight ascending          // O(E log E)

    parent = [0, 1, ..., V-1]
    rank   = [0, 0, ..., 0]                  // union by rank
    totalWeight = 0

    for each (u, v, w) in sorted edges:
        pu = Find(parent, u)
        pv = Find(parent, v)
        if pu == pv: continue

        if rank[pu] < rank[pv]:   parent[pu] = pv
        elif rank[pu] > rank[pv]: parent[pv] = pu
        else:
            parent[pv] = pu
            rank[pu]++

        totalWeight += w
        add (u, v, w) to mstEdges
        if |mstEdges| == V - 1: break
    return totalWeight, mstEdges

Find(parent, x):                              // iterative path compression
    root = x
    while parent[root] != root: root = parent[root]
    while parent[x] != root:                  // compress: point all nodes to root
        next = parent[x]
        parent[x] = root
        x = next
    return root
```

Implementation:

```
public static class KruskalOptimized
{
    1 reference
    public static (double totalWeight, List<(int u, int v, double w)> edges) Run(WeightedGraph graph)
    {
        var edges = graph.GetAllEdges();

        // Sort edges - same cost as classic, but rest of algorithm is faster
        edges.Sort((a, b) => a.w.CompareTo(b.w));

        int n = graph.VertexCount;
        int[] parent = Enumerable.Range(0, n).ToArray();
        int[] rank = new int[n]; // union by rank

        double totalWeight = 0;
        var mstEdges = new List<(int, int, double)>(n - 1);

        foreach (var (u, v, w) in edges)
        {
            int pu = Find(parent, u);
            int pv = Find(parent, v);
            if (pu == pv) continue;

            // Union by rank: attach smaller tree under larger
            if (rank[pu] < rank[pv])
                parent[pu] = pv;
            else if (rank[pu] > rank[pv])
                parent[pv] = pu;
            else
            {
                parent[pv] = pu;
                rank[pu]++;
            }

            totalWeight += w;
            mstEdges.Add((u, v, w));

            if (mstEdges.Count == n - 1) break;
        }

        return (totalWeight, mstEdges);
    }
```

Figure 4: Kruskal Optimized implementation in C# (Part 1)

```
    /// <summary>Iterative path compression (avoids recursive call stack).</summary>
    2 references
    private static int Find(int[] parent, int x)
    {
        int root = x;
        while (parent[root] != root)
            root = parent[root];

        // Path compression: point every node on the path directly to root
        while (parent[x] != root)
        {
            int next = parent[x];
            parent[x] = root;
            x = next;
        }
        return root;
    }
}
```

Figure 5: Kruskal Optimized implementation in C# (Part 2)

Why the Optimized Version is Faster:

- **Union by rank:** Attaching the smaller tree under the larger one during each union operation bounds the maximum tree height at $O(\log V)$, ensuring that no single

`Find` call traverses a long chain. This prevents the degenerate tall-tree scenario that path compression alone cannot avoid if unions are performed in an unfavorable order.

- **Iterative path compression:** Replacing the recursive `Find` with an iterative two-pass version eliminates call-stack frame allocations and return-address tracking on long chains, which is particularly beneficial at large V where Union-Find trees can be deep before compression takes effect.

- **Pre-allocated MST list:** The optimized version initializes the MST edge list with capacity $V - 1$, eliminating the dynamic resizing reallocations that the classic version triggers as edges are added.

- **Limits of optimization:** Because the $O(E \log E)$ sort is the dominant cost at all densities, and both classic and optimized Kruskal perform the same sort, the Union-Find improvements provide only marginal wall-clock speedups. The data confirms this — at most densities Classic and Optimized Kruskal record nearly identical times.

EMPIRICAL ANALYSIS BY GRAPH DENSITY

Each subsection below presents benchmark results (averages of 5 runs) for one graph density level, covering both Prim and Kruskal on sizes from 100 to 3000 vertices. A terminal output figure and two performance plots — one per algorithm family — are included for each density. Graph density is controlled by an edge probability factor: Very Sparse uses 5% of possible edges, Sparse uses 30%, Dense uses 70%, and Very Dense uses 95%, approximating a complete graph.

Very Sparse Graph

A graph with approximately 5% of the maximum possible edges, where most vertices have very few connections. This density level models real-world networks such as road maps and utility grids where connections are rare relative to the population size. With $E \approx 0.05 \cdot \frac{V(V-1)}{2}$, both the Prim heap and the Kruskal edge list remain very small throughout execution.

Results:

```
VerySparse    2000  PrimClassic       12.622 ms
VerySparse    2000  PrimOptimized     2.917 ms
VerySparse    2000  KruskalClassic    16.371 ms
VerySparse    2000  KruskalOptimized  16.929 ms
VerySparse    2500  PrimClassic       20.374 ms
VerySparse    2500  PrimOptimized     4.328 ms
VerySparse    2500  KruskalClassic    26.026 ms
VerySparse    2500  KruskalOptimized  26.724 ms
VerySparse    3000  PrimClassic       29.647 ms
VerySparse    3000  PrimOptimized     5.997 ms
VerySparse    3000  KruskalClassic    38.381 ms
VerySparse    3000  KruskalOptimized  38.998 ms
```

Figure 6: Very Sparse — Terminal Benchmark Output

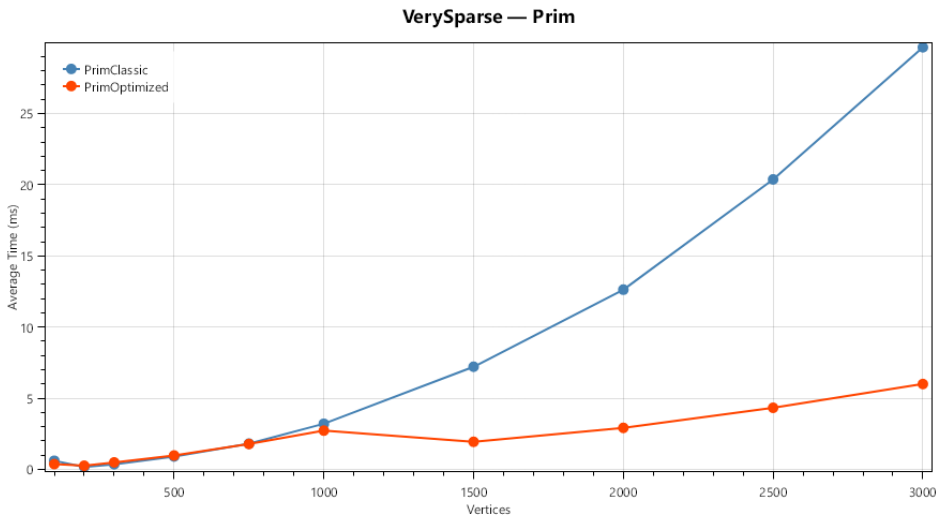

Figure 7: Very Sparse — Prim Performance Plot

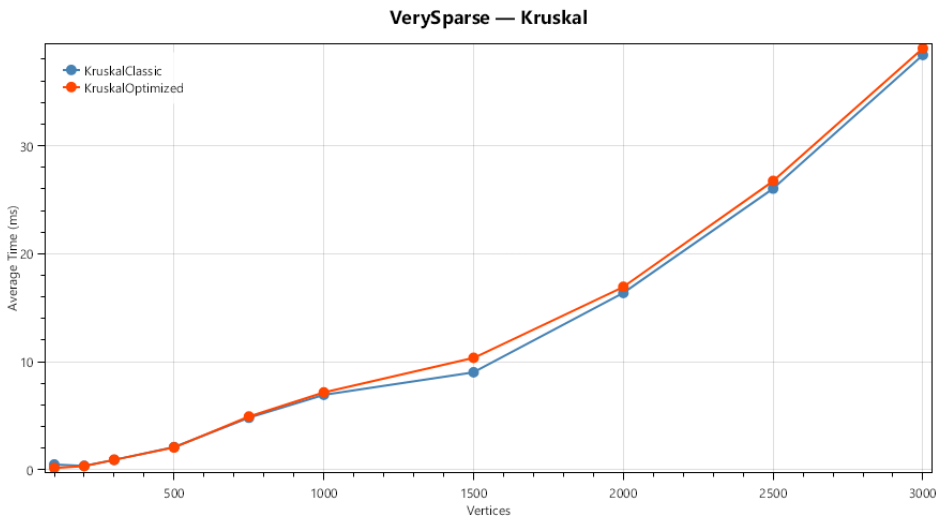

Figure 8: Very Sparse — Kruskal Performance Plot

Analysis: Very sparse graphs reveal the most dramatic separation between Prim Classic and Prim Optimized in the entire benchmark suite. At 3000 vertices, Prim Optimized records just 5.997 ms against Prim Classic's 29.647 ms — a nearly 5× speedup. With only 5% of edges present, the priority queue in the optimized version stays extremely compact throughout execution, keeping each heap extraction cheap and avoiding the $O(V)$ scan that the classic version must perform at every iteration regardless of how few neighbors exist. Kruskal at very sparse density is highly efficient as well: the edge list is small, the sort is fast, and the Union-Find structure processes very few merges. At 3000 vertices, Kruskal Classic records 38.381 ms and Kruskal Optimized 38.998 ms — essentially identical, confirming that the sort cost dominates and Union-Find improvements are negligible when the edge count is low. Kruskal is slightly slower than Prim

Optimized at this density because even a small edge list requires a sort, while Prim's heap processes only reachable edges on demand.

Sparse Graph

A graph with approximately 30% of the maximum possible edges. This density level represents intermediate connectivity such as communication networks or dependency graphs where vertices have a moderate number of links but the graph remains far from complete.

Results:

```
Sparse        2000  PrimClassic       21.498 ms
Sparse        2000  PrimOptimized     12.337 ms
Sparse        2000  KruskalClassic    108.638 ms
Sparse        2000  KruskalOptimized  109.286 ms
Sparse        2500  PrimClassic       33.991 ms
Sparse        2500  PrimOptimized     18.707 ms
Sparse        2500  KruskalClassic    172.770 ms
Sparse        2500  KruskalOptimized  177.841 ms
Sparse        3000  PrimClassic       50.343 ms
Sparse        3000  PrimOptimized     27.273 ms
Sparse        3000  KruskalClassic    254.949 ms
Sparse        3000  KruskalOptimized  256.972 ms
```

Figure 9: Sparse — Terminal Benchmark Output

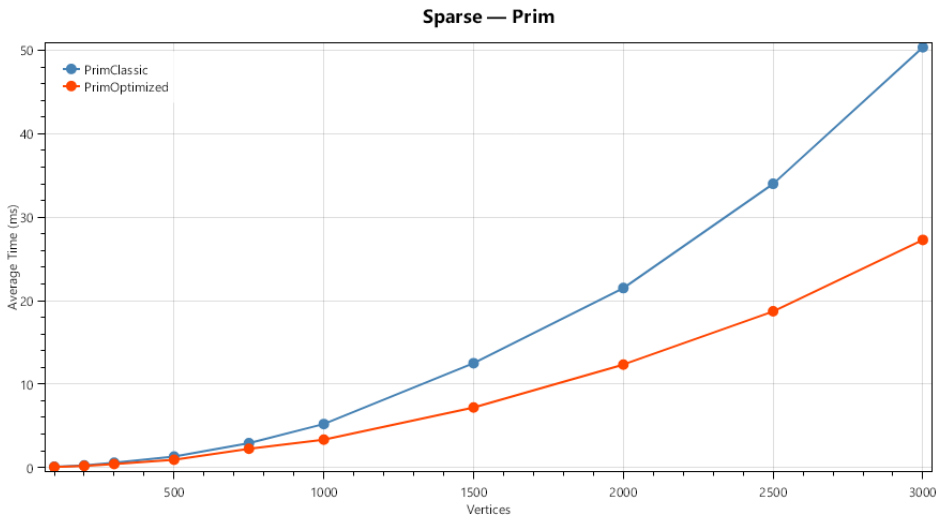

Figure 10: Sparse — Prim Performance Plot

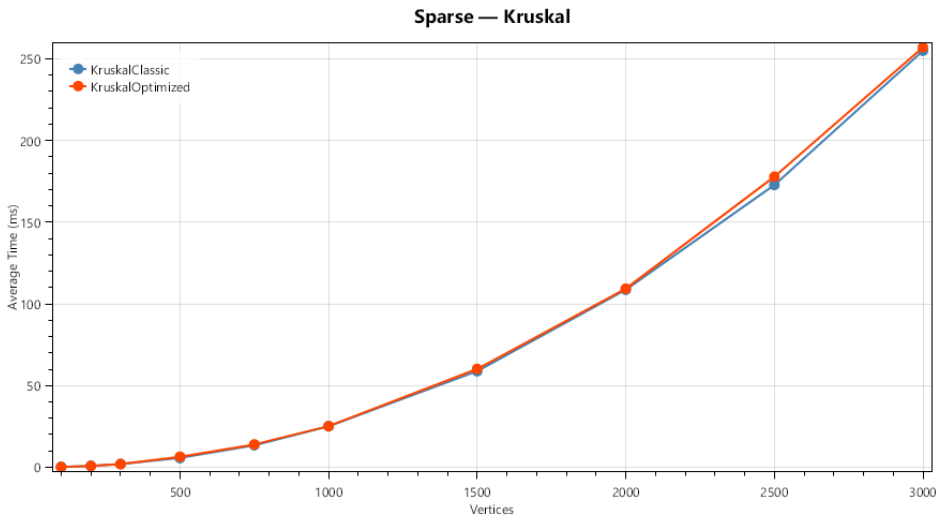

Figure 11: Sparse — Kruskal Performance Plot

Analysis: At 30% density the Prim Optimized advantage remains substantial but begins to narrow compared to the very sparse case. At 3000 vertices, Prim Optimized records 27.273 ms versus Prim Classic's 50.343 ms — roughly a 1.85× speedup. The growing edge count means more relaxations occur during traversal, causing the priority queue to accumulate more entries and increasing the total number of $O(\log V)$ heap operations. Kruskal's behavior shifts more noticeably at this density: the edge list grows to approximately 30% of $\frac{V(V-1)}{2}$ entries, making the sort considerably more expensive. At 3000 vertices, Kruskal Classic records 254.949 ms and Kruskal Optimized 256.972 ms — again nearly indistinguishable, with the sort completely dominating both variants. The gap between Prim Optimized and Kruskal has widened substantially compared to very sparse density, illustrating that Kruskal's $O(E \log E)$ sort cost grows much faster than Prim's heap-based traversal as edges are added.

Dense Graph

A graph with approximately 70% of the maximum possible edges, representing highly interconnected networks. At this density, each vertex connects to the majority of other vertices, significantly increasing both the Kruskal edge list size and the number of relaxations Prim must process per vertex.

Results:

```
Dense        2000  PrimClassic       36.039 ms
Dense        2000  PrimOptimized     26.813 ms
Dense        2000  KruskalClassic    266.840 ms
Dense        2000  KruskalOptimized  269.007 ms
Dense        2500  PrimClassic       56.746 ms
Dense        2500  PrimOptimized     41.726 ms
Dense        2500  KruskalClassic    422.899 ms
Dense        2500  KruskalOptimized  427.952 ms
Dense        3000  PrimClassic       81.840 ms
Dense        3000  PrimOptimized     58.635 ms
Dense        3000  KruskalClassic    622.421 ms
Dense        3000  KruskalOptimized  630.077 ms
```

Figure 12: Dense — Terminal Benchmark Output

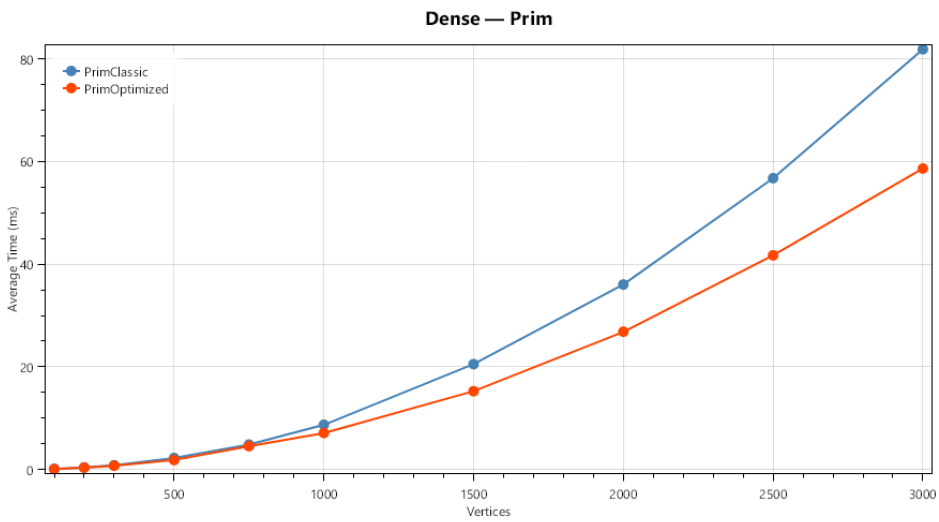

Figure 13: Dense — Prim Performance Plot

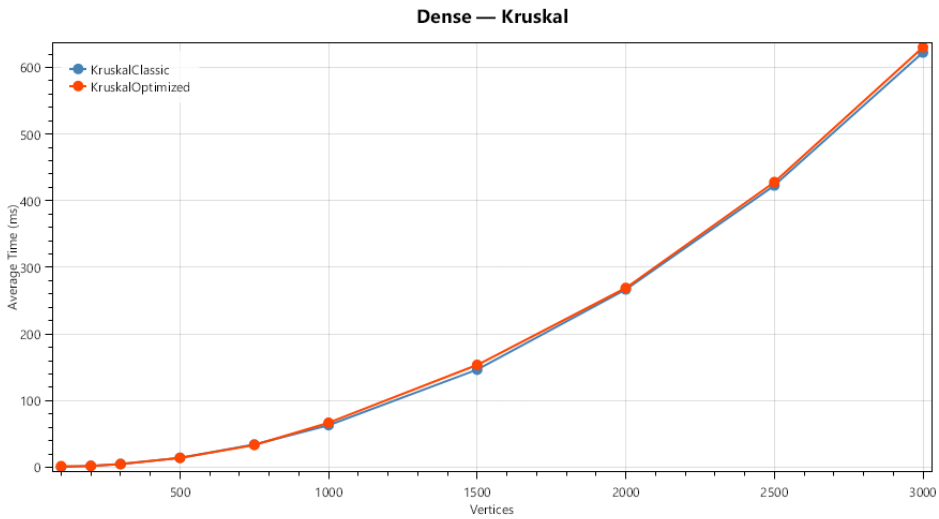

Figure 14: Dense — Kruskal Performance Plot

Analysis: Dense graphs reveal the point at which Kruskal's sort-dominated cost becomes very expensive and Prim's heap advantage begins to erode relative to its own classic version. At 3000 vertices, Prim Classic records 81.840 ms and Prim Optimized 58.635 ms — a 1.4× gap, noticeably narrower than at sparse density. The growing heap size at 70% density means more stale entries are processed and more $O(\log V)$ operations accumulate, gradually closing the gap with the flat linear scan. Kruskal at this density is dramatically more expensive: Kruskal Classic records 622.421 ms and Kruskal Optimized 630.077 ms at 3000 vertices. The near-identical times for classic and optimized Kruskal confirm once again that the sort cost is the sole driver of runtime, and the Union-Find improvements are invisible in wall-clock measurements. The order-of-magnitude difference between Prim Optimized (58.635 ms) and Kruskal (over 620 ms) at this density makes clear that Kruskal is an unsuitable choice for dense graphs.

Very Dense Graph

A graph with approximately 95% of the maximum possible edges, approaching a complete graph. This represents the theoretical worst case for Kruskal in terms of edge list size and sort cost, and the most challenging scenario for Prim's priority queue in terms of stale-entry accumulation.

Results:

```
VeryDense     2000  PrimClassic       44.737 ms
VeryDense     2000  PrimOptimized     35.981 ms
VeryDense     2000  KruskalClassic    366.120 ms
VeryDense     2000  KruskalOptimized  376.220 ms
VeryDense     2500  PrimClassic       69.896 ms
VeryDense     2500  PrimOptimized     55.402 ms
VeryDense     2500  KruskalClassic    584.197 ms
VeryDense     2500  KruskalOptimized  591.657 ms
VeryDense     3000  PrimClassic       102.453 ms
VeryDense     3000  PrimOptimized     78.623 ms
VeryDense     3000  KruskalClassic    857.455 ms
VeryDense     3000  KruskalOptimized  873.234 ms
```

Figure 15: Very Dense — Terminal Benchmark Output

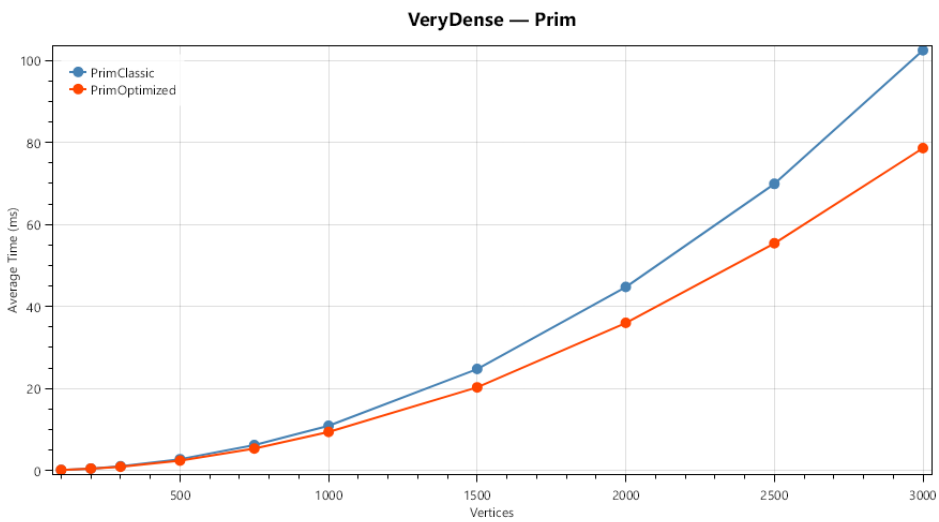

Figure 16: Very Dense — Prim Performance Plot

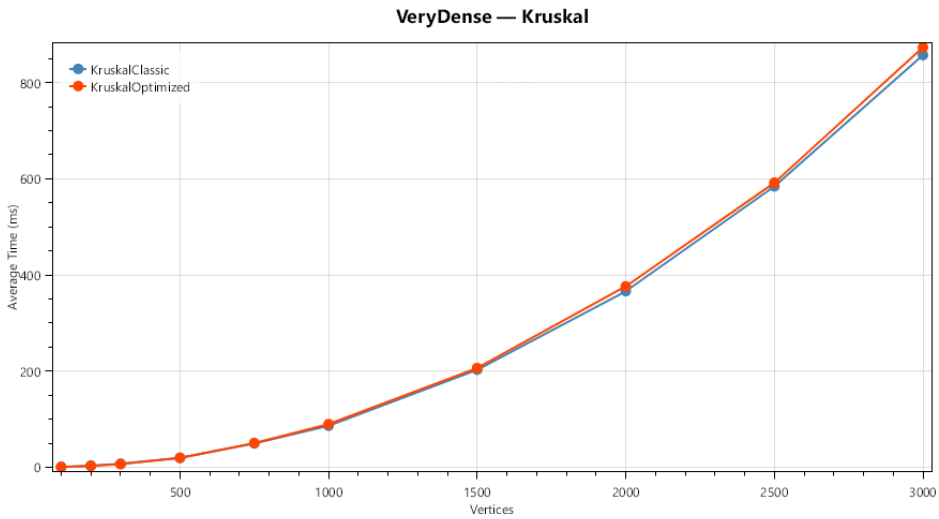

Figure 17: Very Dense — Kruskal Performance Plot

Analysis: At 95% density the performance trends established at lower densities reach their logical conclusion. Prim Classic records 102.453 ms and Prim Optimized 78.623 ms at 3000 vertices — a gap of roughly 1.3×, the narrowest observed across all densities. The priority queue is flooded with stale entries at near-complete density, and the cumulative $O(\log V)$ extraction overhead approaches the cost of the flat array scan. Kruskal at this density is catastrophic by comparison: Kruskal Classic records 857.455 ms and Kruskal Optimized 873.234 ms at 3000 vertices. The optimized variant is marginally *slower* than classic at this size — an anomaly explained by the slight overhead of the rank array write operations and iterative path compression bookkeeping on the extremely large Union-Find structure, when the sort already dominates so completely that no Union-Find improvement can matter. The contrast between Prim Optimized at 78.623 ms and Kruskal at over 857 ms at 3000 vertices is the starkest finding of the entire study, confirming that Kruskal should not be used on dense graphs where E approaches V^2.

OVERALL RESULTS

The overall results reveal fundamentally different scaling behavior between Prim and Kruskal as graph density increases, driven by their contrasting algorithmic structures. Prim's runtime is governed by vertex count and data structure efficiency, while Kruskal's runtime is governed by edge count through the mandatory sort.

Prim Optimized is the strongest general-purpose MST algorithm across all tested densities, achieving the lowest times in every single benchmark configuration. Its advantage over Prim Classic is greatest at very sparse density — 5.997 ms versus 29.647 ms at 3000 vertices, a 5× speedup — and gradually narrows to approximately 1.3× at very dense density (78.623 ms versus 102.453 ms at 3000 vertices). This density-dependent erosion of the heap's advantage follows directly from the increasing number of stale entries that accumulate in the priority queue as edge count grows, raising the total number of $O(\log V)$ operations to levels approaching the cost of the $O(V)$ linear scan.

Prim Classic scales predictably and competitively at high densities. On very dense graphs at 3000 vertices it records 102.453 ms — less than twice the optimized variant — making it a pragmatically reasonable choice when implementation simplicity is valued and the graph is known to be dense. Its $O(V^2)$ complexity is independent of edge count, which means its runtime does not grow as steeply as Kruskal's when density increases.

Kruskal Classic and Kruskal Optimized are effectively equivalent at every density and size tested. The times differ by less than 2% in nearly all configurations — for example, at 3000 vertices on very dense graphs, Classic records 857.455 ms and Optimized 873.234 ms. This near-perfect parity across all conditions confirms that the $O(E \log E)$ sort cost completely dominates the total runtime, and no Union-Find improvement — however theoretically sound — can produce a measurable wall-clock benefit when the sort is the bottleneck. The practical implication is that the classic Kruskal implementation is sufficient whenever Kruskal is the appropriate algorithm choice, as the optimized Union-Find provides no observable benefit.

Kruskal versus Prim presents the most striking finding of the study. At very sparse density Kruskal is competitive with Prim Classic (38.381 ms vs 29.647 ms at 3000 vertices), since the small edge list is sorted quickly. However, as density increases, Kruskal's times grow dramatically — from 38.381 ms at very sparse to 857.455 ms at very dense density at 3000 vertices, a 22× increase. Prim Classic over the same density range grows from 29.647 ms to only 102.453 ms, a 3.5× increase. This asymmetry confirms that Kruskal is exclusively suitable for sparse graphs and should be avoided as edge density grows toward completeness.

CONCLUSION

This empirical study of Prim and Kruskal across four graph density levels produces concrete, data-backed conclusions that both confirm theoretical predictions and expose important practical nuances invisible to asymptotic analysis alone.

The most significant finding is that the two algorithms occupy fundamentally different density niches. Kruskal is an edge-driven algorithm whose runtime is governed by the mandatory $O(E \log E)$ sort. At very sparse density this sort is cheap and Kruskal is competitive, but as density grows toward 95%, the edge list approaches $O(V^2)$ entries and the sort becomes catastrophically expensive — over 857 ms at 3000 vertices versus Prim Optimized's 78 ms at the same configuration. No Union-Find optimization can compensate for this, and the benchmark data confirms it: Kruskal Classic and Kruskal Optimized are indistinguishable in wall-clock time at every tested density, differing by at most 2%. The Union-Find improvements are theoretically sound but empirically irrelevant when sorting dominates.

Prim, by contrast, is a vertex-driven algorithm whose runtime depends on how efficiently the minimum-weight cut edge can be identified at each step. The heap-based optimized version delivers its most decisive advantage at very sparse density — nearly 5× faster than the classic linear scan at 3000 vertices — because the priority queue stays small and each extraction is cheap. As density increases, stale heap entries accumulate and the logarithmic overhead per extraction erodes the advantage, narrowing the gap to roughly 1.3× at very dense density. Importantly, Prim Optimized never loses to Prim Classic in the collected data, confirming that the heap-based approach is always at least as good and often substantially better.

The greedy principle shared by both algorithms guarantees MST correctness, but the choice of greedy strategy — growing a single tree from a seed vertex versus globally sorting and merging edge-disjoint components — produces dramatically different empirical performance profiles when graph density varies. This study demonstrates that algorithm selection for MST computation must be guided by the input graph's density: Prim Optimized is the unconditional recommendation for dense and very dense graphs, while Kruskal remains viable only on sparse inputs where $E \ll V^2$.

In conclusion, for general-purpose MST computation on graphs of unknown or variable density, Prim Optimized is the recommended implementation. For sparse graphs where edge count is guaranteed to be low, Kruskal Classic and Kruskal Optimized are equivalent choices and either can be used, with the classic version preferred for its simplicity given that the optimized Union-Find provides no measurable benefit. Prim Classic is a reasonable fallback on very dense graphs when implementation simplicity is preferred, as the heap's advantage narrows considerably at high edge density.

REFERENCES

[1] Moisei, D. GitHub Repository. https://github.com/Moisei-D/Algorithm-Analysis

[2] GeeksforGeeks. (2024). *Prim's Algorithm for Minimum Spanning Tree.* Retrieved April 2026, from https://www.geeksforgeeks.org/prims-minimum-spanning-tree-mst-greedy

[3] GeeksforGeeks. (2024). *Kruskal's Minimum Spanning Tree Algorithm.* Retrieved April 2026, from https://www.geeksforgeeks.org/kruskals-minimum-spanning-tree-algorit

[4] Scaler. (2024). *Prim's and Kruskal's Algorithm.* Retrieved April 2026, from https://www.scaler.com/topics/prims-and-kruskal-algorithm/